

クラス設計ドキュメント

対象: PortfolioPiece / Scripts (C++) 作成日: 2026-01-31

1. 目的

プレイヤー操作、敵生成、UIやカメラ制御を含むゲーム進行のクラス構成を整理し、各クラスの責務と依存関係を明確化する。
これにより、拡張（敵種類追加や演出強化）と保守（バグ切り分けや調整）を容易にする。

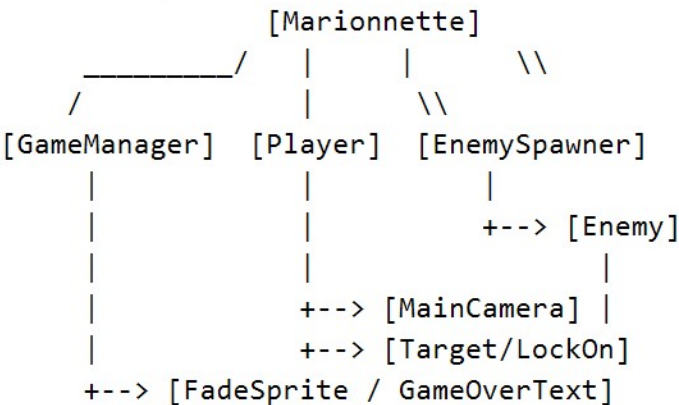
2. 設計概要

- ・各ゲーム要素は Marionnette を基底とするスクリプトとして実装する。
- ・ゲーム進行の制御は GameManager が担当し、UIやシーン遷移を統括する。
- ・プレイヤー操作は Player、敵管理は EnemySpawner と Enemy が担う。
- ・視点制御は MainCamera が担当し、プレイヤーに追従する。
- ・UIやロックオンなどは専用クラス（Target/LockOn など）に分離する。

3. 採用理由・目的

- ・責務分離: 操作、敵生成、演出、UIを分割し、変更の影響範囲を局所化。
- ・拡張性: EnemySpawner/Enemy の組でフェーズ追加や攻撃パターン変更を容易化。
- ・可読性: Marionnette に統一することでライフサイクル（Start/Update）を揃える。
- ・調整容易性: 各クラスにパラメータを集約し、チューニング箇所を明確化。

4. クラス図 (ASCII)



依存関係: GameManager -> Player/UI, EnemySpawner -> Enemy,
Player -> MainCamera/Target, Enemy -> Player/Target/LockOn

5. 主要クラス解説

- ・GameManager: フェード演出やゲーム終了演出の制御を一元化。
- ・Player: 移動/回避/攻撃/被弾状態の管理と UI 連動。
- ・EnemySpawner / Enemy: 生成フェーズ管理と攻撃パターン制御。
- ・MainCamera: 追従とイントロ演出のブレンド。

6. 今後の拡張余地

- ・Enemy の攻撃パターン追加は enum と UpdateAttack の拡張で対応。
- ・GameManager に難易度管理を追加し EnemySpawner のフェーズ調整へ反映。
- ・UI 系クラスを View/Model で分割すると保守性が向上。

出典: Assets/Scripts 以下の GameManager, Player, Enemy, EnemySpawner, MainCamera を基に整理。